UNIT – III Design concepts: Design within the context of software engineering, Design process, Design concepts, Transform mapping, Transaction mapping, Design documentation.

Design concepts:

Software design encompasses the set of principles, concepts, and practices that lead to the development of a high-quality system or product.

Design principles establish an overriding philosophy that guides you in the design work you must perform.

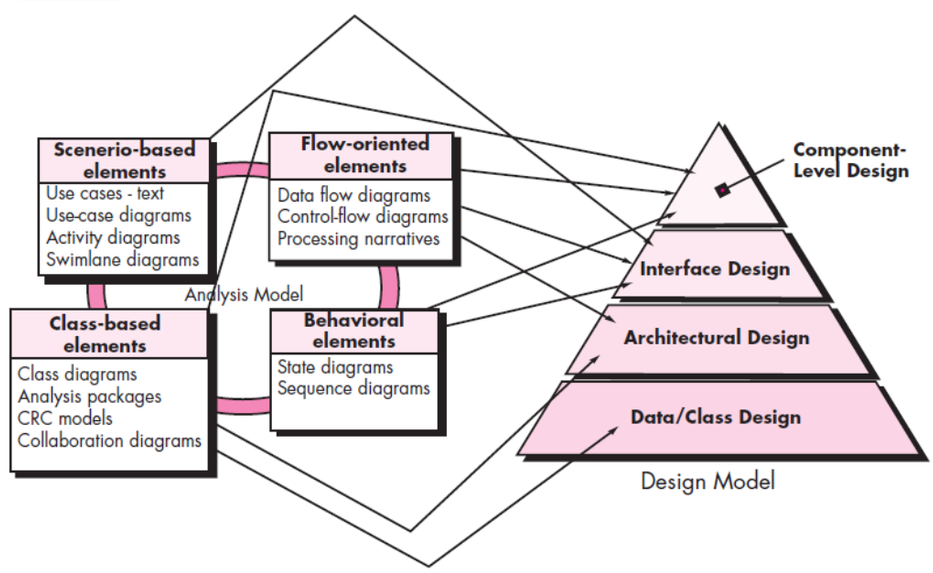
Design concepts must be understood before the mechanics of design practice are applied, and design practice itself leads to the creation of various representations of the software that serve as a guide for the construction activity that follows.

Design within the Context of Software Engineering

Software design sits at the technical kernel of software engineering and is applied regardless of the software process model that is used.

Beginning once software requirements have been analyzed and modeled, software design is the last software engineering action within the modeling activity and sets the stage for construction (code generation and testing).

FIGURE 8.1 Translating the requirements model into the design model



The flow of information during software design is illustratedin Figure 8.1.

The requirements model, manifested by scenario-based, class-based, flow-oriented, and behavioral elements, feed the design task.

Using design notation and design methods discussed in later chapters, design produces a data/class design, an architectural design, an interface design, and a component design.

The data/class design transforms class models into design class realizations and the requisite data structures required to implement the software.

The objects and relationships defined in the CRC diagram and the detailed data content depicted by class attributes and other notation provide the basis for the data designation.

Part of class design may occur in conjunction with the design of software architecture. More detailed class design occurs as each software component is designed.

The architectural design defines the relationship between major structural elements of the software, the architectural styles and design patterns that can be used to achieve the requirements defined for the system, and the constraints that affect the way in which architecture can be implemented.

The architectural design representation—the framework of a computer-based system—is derived from the requirements model.

The interface design describes how the software communicates with systems that interoperate with it, and with humans who use it.

An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior. Therefore, usage scenarios and behavioral models provide much of the information required for interface design.

The component-level design transforms structural elements of the software architecture into a procedural description of software components. Information obtained from the class-based models, flow models, and behavioral models serve as the basis for component design.

During design you make decisions that will ultimately affect the success of software construction and, as important, the ease with which software can be maintained.

But why is design so important?
The importance of software design can be stated with a single word—quality.

Design is the place where quality is fostered in software engineering. Design provides you with representations of software that can be assessed for quality. Design is the only way that you can accurately translate stakeholder's requirements into a finished software product or system.

Software design serves as the foundation for all the software engineering and software support activities that follow. Without design, you risk building an unstable system—one that will fail when small changes are made; one that may be difficult to test; one whose quality cannot be assessed until late in the software process, when time is short and many dollars have already been spent.

THE DESIGN PROCESS

Software design is an iterative process through which requirements are translated into a "blueprint" for constructing the software.
Initially, the blueprint depicts a holistic view of software.

That is, the design is represented at a high level of abstraction—a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements.

As design iterations occur, subsequent refinement leads to design representations at much lower levels of abstraction. These can still be traced to requirements, but the connection is more subtle.

Software Quality Guidelines and Attributes:

Throughout the design process, the quality of the evolving design is McGlaughlin [McG91] suggests three characteristics that serve as a guide for the evaluation of a good design:

• The design must implement all of the explicit requirements contained in the requirements model, and it must accommodate all of the implicit requirements desired by stakeholders.

• The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.

• The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Quality Guidelines

In order to evaluate the quality of a design representation, you and other members of the software team must establish technical criteria for good design.
design concepts that also serve as software quality criteria. For the time being, consider the following guidelines:

1. A design should exhibit an architecture that
   (1) has been created using recognizable architectural styles or patterns,
   (2) is composed of components that exhibit good design characteristics, and
   (3) can be implemented in an evolutionary fashion, facilitating implementation and testing.

2. A design should be modular; that is, the software should be logically partitioned into elements or subsystems.

3. A design should contain distinct representations of data, architecture, interfaces, and components.

4. A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

5. A design should lead to components that exhibit independent functional characteristics.

6. A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.

7. A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.

8. A design should be represented using a notation that effectively communicates its meaning.

These design guidelines are not achieved by chance. They are achieved through the application of fundamental design principles, systematic methodology, and thorough review.

Quality Attributes.

Hewlett-Packard [Gra87] developed a set of software quality attributes that has been given the acronym FURPS—functionality, usability, reliability, performance, and supportability. The FURPS quality attributes represent a target for all software design:

• **Functionality** is assessed by evaluating the feature set and capabilities of the program, the generality of the functions that are delivered, and the security of the overall system.
Usability is assessed by considering human factors, overall
aesthetics, consistency, and documentation.

• **Reliability** is evaluated by measuring the frequency and severity of failure, the accuracy of output results, the mean-time-to-failure (MTTF), the ability to recover from failure, and the predictability of the program.

• **Performance** is measured by considering processing speed, response time, resource consumption, throughput, and efficiency.

• **Supportability** combines the ability to extend the program (extensibility),adaptability, serviceability—these three attributes represent a more common term, maintainability—and in addition, testability, compatibility, configurability(the ability to organize and control elements of the software configuration, the ease with which a system can be installed, and the ease with which problems can be localized.
Not every software quality attribute is weighted equally as the software design is developed.
One application may stress functionality with a special emphasis on security. Another may demand performance with particular emphasis on processing speed. A third might focus on reliability. Regardless of the weighting, it is important to note that these quality attributes must be considered as design commences, not after the design is complete and construction has begun.

The Evolution of Software Design

The evolution of software design is a continuing process that has now spanned almost six decades. Early design work concentrated on criteria for the development of modular programs [Den73] and methods for refining software structures in a top down manner [Wir71]. Procedural aspects of design definition evolved into a philosophy called structured programming [Dah72], [Mil72]. Later work proposed methods for the translation of data flow [Ste74] or data structure (e.g., [Jac75], [War74]) into design definition.

A number of design methods, growing out of the work just noted, are being applied throughout the industry. Like the analysis methods presented in Chapters 6 and7, each software design method introduces unique heuristics and notation, as well as a somewhat parochial view of what characterizes design quality.

Yet, all of these methods have a number of common characteristics:
(1) a mechanism for the translation of the requirements model into a design representation,
(2) a notation for representing functional components and their interfaces,
(3) Heuristics for refinement and partitioning, and
(4) Guidelines for quality assessment.

Regardless of the design method that is used, you should apply a set of basic concepts to data, architectural, interface, and component-level design. These concepts are considered in the sections that follow.

DESIGN CONCEPTS

A set of fundamental software design concepts has evolved over the history of software engineering. Although the degree of interest in each concept has varied over the years, each has stood the test of time. Each provides the software designer with foundation from which more sophisticated design methods can be applied. Each helps you answer the following questions:

• What criteria can be used to partition software into individual components?
• How is function or data structure detail separated from a conceptual representation of the software?
• What uniform criteria define the technical quality of a software design?
M. A. Jackson [Jac75] once said: "The beginning of wisdom for a [software engineer] is to recognize the difference between getting a program to work, and getting it right." Fundamental software design concepts provide the necessary framework for "getting it right." In the sections that follow, I present a brief overview of important software design concepts that span both traditional and object-oriented software development.

1 .Abstraction

When you consider a modular solution to any problem, many levels of abstraction can be posed. At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment. At lower levels of abstraction, a more detailed description of the solution is provided. Problem-oriented terminology is coupled with implementation-oriented terminology in an effort to state a solution.

Finally, at the lowest level of abstraction, the solution is stated in a manner that can be directly implemented.

As different levels of abstraction are developed, you work to create both procedural and data abstractions.

A *procedural abstraction* refers to a sequence of instructions that have a specific and limited function. The name of a procedural abstraction implies these functions, but specific details are suppressed. An example of a procedural abstraction would be the word *open* for a door. *Open* implies a long sequence of procedural steps (e.g., walk to the door, reach out and grasp knob, turn knob and pull door, step away from moving door, etc.).

A *data abstraction* is a named collection of data that describes a data object. In the context of the procedural abstraction *open,* we can define a data abstraction called **door.** Like any data object, the data abstraction for **door** would encompass set of attributes that describe the door (e.g., door type, swing direction, opening mechanism, weight, dimensions). It follows that the procedural abstraction *open* would make use of information contained in the attributes of the data abstraction *door.*

2.Architecture

Software architecture alludes to "the overall structure of the software and the ways in which that structure provides conceptual integrity for a system" [Sha95a]. In its simplest form, architecture is the structure or organization of program components(modules), the manner in which these components interact, and the structure of data that are used by the components. In a broader sense, however, components can be generalized to represent major system elements and their interactions.

One goal of software design is to derive an architectural rendering of a system. This rendering serves as a framework from which more detailed design activities are conducted. A set of architectural patterns enables a software engineer to solve common design problems.

Shaw and Garlan [Sha95a] describe a set of properties that should be specified apart of an architectural design:

Structural properties : This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another. For example, objects are packaged to encapsulate both data and the processing that manipulates the data and interactive the invocation of methods.

Extra-functional properties: The architectural design description should address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

Families of related systems: The architectural design should draw upon repeatable patterns that are commonly encountered in the design of families of similar systems. In essence, the design should have the ability to reuse architectural building blocks.

Given the specification of these properties, the architectural design can be represented using one or more of a number of different models [Gar95]. Structural models represent architecture as an organized collection of program components.
Framework models increase the level of design abstraction by attempting to identify repeatable architectural design frameworks that are encountered in similar types of applications. Dynamic models address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function of external events. Process models focus on the design of the business or technical process that the system must accommodate. Finally, functional models can be used to represent the functional hierarchy of a system.

A number of different architectural description languages (ADLs) have been developed to represent these models [Sha95b]. Although many different ADLs have been proposed, the majority provide mechanisms for describing system components and the manner in which they are connected to one another.

You should note that there is some debate about the role of architecture in design. Some researchers argue that the derivation of software architecture should be separated from design and occurs between requirements engineering actions and more conventional design actions. Others believe that the derivation of architecture is an integral part of the design process.

3 .Patterns

Brad Appleton defines a *design pattern* in the following manner: “A pattern is named nugget of insight which conveys the essence of a proven solution to a recurring problem within a certain context amidst competing concerns” [App00]. Stated in another way, a design pattern describes a design structure that solves a particular design problem within a specific context and amid “forces” that may have an impact on the manner in which the pattern is applied and used.

The intent of each design pattern is to provide a description that enables a designer to determine (1) whether the pattern is applicable to the current work,(2) whether the pattern can be reused (hence, saving design time), and (3) whether the pattern can serve as a guide for developing a similar, but functionally or structurally different pattern.

4. Separation of Concerns

Separation of concerns is a design concept [Dij82] that suggests that any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently. A *concern* is a feature or behavior that is specified as part of the requirements model for the software. By separating concerns into smaller, and therefore more manageable pieces, a problem takes less effort and time to solve.

For two problems, *p*1 and *p*2, if the perceived complexity of *p*1 is greater than the perceived complexity of *p*2, it follows that the effort required to solve *p*1 is greater than the effort required to solve *p*2. As a general case, this result is intuitively obvious.
It does take more time to solve a difficult problem.
It also follows that the perceived complexity of two problems when they are combined is often greater than the sum of the perceived complexity when each is taken separately. This leads to a divide-and-conquer strategy—it's easier to solve a complex problem when you break it into manageable pieces. This has important implications with regard to software modularity.

Separation of concerns is manifested in other related design concepts: modularity, aspects, functional independence, and refinement. Each will be discussed in the subsections that follow.

5. Modularity

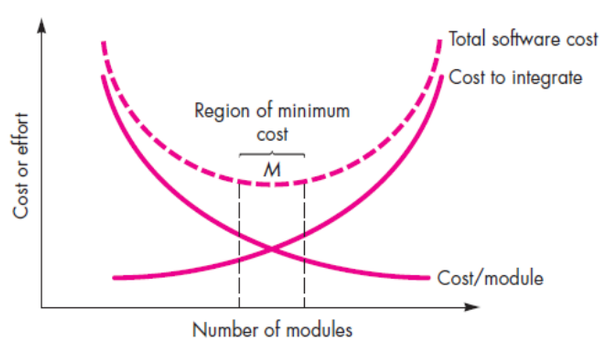
Modularity is the most common manifestation of separation of concerns. Software is divided into separately named and addressable components, sometimes called *modules,* that are integrated to satisfy problem requirements.

It has been stated that "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78]. Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer.

The number of control paths, span of reference, number of variables, and overall complexity would make understanding close to impossible. In almost all instances, you should break the design into many modules, hoping to make understanding easier and, as a consequence, reduce the cost required to build the software.

Recalling my discussion of separation of concerns, it is possible to conclude that if you subdivide software indefinitely the effort required to develop it will become negligibly small! Unfortunately, other forces come into play, causing this conclusion to be (sadly) invalid. Referring to Figure 8.2, the effort (cost) to develop an individual software module does decrease as the total number of modules increases.

FIGURE 8.2
Modularity and software cost



Given the same set of requirements, more modules means smaller individual size. However, as the number of modules grows, the effort (cost) associated with integrating the modules also grows. These characteristics lead to a total cost or effort curve shown in the figure. There is a number, *M,* of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict *M* with assurance.

The curves shown in Figure 8.2 do provide useful qualitative guidance when modularity is considered. You should modularize, but care should be taken to stay in the vicinity of *M.* Under modularity or over modularity should be avoided. But how do you know the vicinity of *M*? How modular should you make software? The answers to these questions require an understanding of other design concepts considered later in this chapter.

6. Information Hiding

The concept of modularity leads you to a fundamental question: "How do I decompose software solution to obtain the best set of modules?" The principle of information hiding [Par72] suggests that modules be "characterized by design decisions that (each) hides from all others." In other words, modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information.

Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function. Abstraction helps to define the procedural (or informational) entities that make up the software. Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module [Ros75].

7. Functional Independence

The concept of functional independence is a direct outgrowth of separation of concerns, modularity, and the concepts of abstraction and information hiding. In landmark papers on software design, Wirth [Wir71] and Parnas [Par72] allude to refinement techniques that enhance module independence. Later work by Stevens, Myers, and Constantine [Ste74] solidified the concept.

Functional independence is achieved by developing modules with "single minded "function and an "aversion" to excessive interaction with other modules.

Stated another way, you should design software so that each module addresses specific subset of requirements and has a simple interface when viewed from other parts of the program structure. It is fair to ask why independence is important.

Software with effective modularity, that is, independent modules, is easier to develop because function can be compartmentalized and interfaces are simplified(consider the ramifications when development is conducted by a team). Independent modules are easier to maintain (and test) because secondary effects caused by designer code modification are limited, error propagation is reduced, and reusable modules are possible.

To summarize, functional independence is a key to good design, and design is the key to software quality.

Independence is assessed using two qualitative criteria: cohesion and coupling.

Cohesion is an indication of the relative functional strength of a module.

Coupling isan indication of the relative interdependence among modules.

Cohesion is a natural extension of the information-hiding concept .A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing. Although you should always strive for high cohesion(i.e., single-mindedness), it is often necessary and advisable to have software component perform multiple functions. However, "schizophrenic" components(modules that perform many unrelated functions) are to be avoided if a good design is to be achieved.

Coupling is an indication of interconnection among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.

In software design, you should strive for the lowest possible coupling. Simple connectivity among modules results in software that is easier to understand and less prone to a "ripple effect" [Ste74], caused when errors occur at one location and propagate throughout a system.

8. Refinement

Stepwise refinement is a top-down design strategy originally proposed by NiklausWirth [Wir71]. A program is developed by successively refining levels of procedural detail. A hierarchy is developed by decomposing a macroscopic statement of function (a procedural abstraction) in a stepwise fashion until programming language statements are reached.

Refinement is actually a process of *elaboration.* You begin with a statement of function (or description of information) that is defined at a high level of abstraction.

That is, the statement describes function or information conceptually but provides no information about the internal workings of the function or the internal structure of the information. You then elaborate on the original statement, providing more and more detail as each successive refinement (elaboration) occurs.

Abstraction and refinement are complementary concepts. Abstraction enables you to specify procedure and data internally but suppress the need for "outsiders" to have knowledge of low-level details. Refinement helps you to reveal low-level details as design progresses. Both concepts allow you to create a complete design model as the design evolves.

9. Aspects

As requirements analysis occurs, a set of "concerns" is uncovered. These concerns "include requirements, use cases, features, data structures, quality-of-service issues, variants, intellectual property boundaries, collaborations, patterns and contracts"[AOS07]. Ideally, a requirements model can be organized in a way that allows you to isolate each concern (requirement) so that it can be considered independently. In practice, however, some of these concerns span the entire system and cannot be easily compartmentalized.

As design begins, requirements are refined into a modular design representation. Consider two requirements, *A* and *B.* Requirement *A crosscuts* requirement *B* "if software decomposition [refinement] has been chosen in which *B* cannot be satisfied without taking *A* into account" [Ros04].

For example, consider two requirements for the **SafeHomeAssured.com** WebApp. Requirement *A* is described via the **ACS-DCV** use case discussed in Chapter 6. Design refinement would focus on those modules that would enable a registered user to access video from cameras

placed throughout a space. Requirement *B* is a generic security requirement that states that *a registered user must be validated prior to using***SafeHomeAssured.com.** This requirement is applicable for all functions that are available to registered *SafeHome*users. As design refinement occurs, *A** is a design representation for requirement *A* and *B** is a design representation for requirement *B*.

Therefore, *A** and *B** are representations of concerns, and *B* crosscuts A**.An *aspect* is a representation of a crosscutting concern. Therefore, the design representation***, of the requirement *a registered user must be validated prior to using***SafeHomeAssured.com,** is an aspect of the *SafeHome*WebApp. It is important to identify aspects so that the design can properly accommodate them as refinement and modularization occur. In an ideal context, an aspect is implemented as a separate module (component) rather than as software fragments that are "scattered" or "tangled" throughout many components [Ban06]. To accomplish this, the design architecture should support a mechanism for defining an aspect—a module that enables the concern to be implemented across all other concerns that it crosscuts.

10. Refactoring

An important design activity suggested for many agile methods (Chapter 3),*refactoring* is a reorganization technique that simplifies the design (or code) of component without changing its function or behavior. Fowler [Fow00] defines refactoring in the following manner: "Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design]yet improves its internal structure."

When software is refectories, the existing design is examined for redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failure that can be corrected to yield better design. For example, a first design iteration might yield a component that exhibits low cohesion (i.e., it performs three functions that have only limited relationship to one another). After careful consideration, you may decide that the component should be refectories into three separate components, each exhibiting high cohesion.

The result will be software that is easier to integrate, easier to test, and easier to maintain.

11. Object-Oriented Design Concepts

The object-oriented (OO) paradigm is widely used in modern software engineering. Appendix 2 has been provided for those readers who may be unfamiliar with OOdesign concepts such as classes and objects, inheritance, messages, and polymorphism, among others.

12. Design Classes

The requirements model defines a set of analysis classes (Chapter 6). Each describes some element of the problem domain, focusing on aspects of the problem that arouser visible. The level of abstraction of an analysis class is relatively high.
As the design model evolves, you will define a set of *design classes* that refine the analysis classes by providing design detail that will enable the classes to be implemented, and implement a software infrastructure that supports the business solution.

Five different types of design classes, each representing a different layer of the design architecture, can be developed [Amb01]:

- *User interface classes* define all abstractions that are necessary for human computer interaction (HCI). In many cases, HCI occurs within the context ofa *metaphor* (e.g., a checkbook, an order form, a fax machine), and the design classes for the interface may be visual representations of the elements of the metaphor.

- *Business domain classes* are often refinements of the analysis classes definedearlier. The classes identify the attributes and services (methods) that arerequired to implement some element of the business domain.

- *Process classes* implement lower-level business abstractions required to fullymanage the business domain classes.

- *Persistent classes* represent data stores (e.g., a database) that will persistbeyond the execution of the software.

- *System classes* implement software management and control functions thatenable the system to operate and communicate within its computing environmentand with the outside world.

As the architecture forms, the level of abstraction is reduced as each analysis classis transformed into a design representation. That is, analysis classes represent dataobjects (and associated services that are applied to them) using the jargon of thebusiness domain. Design classes present significantly more technical detail as aguide for implementation.

Arlow and Neustadt [Arl02] suggest that each design class be reviewed to ensurethat it is "well-formed." They define four characteristics of a well-formed designclass:

Complete and sufficient. A design class should be the complete encapsulationof all attributes and methods that can reasonably be expected (basedon a knowledgeable interpretation of the class name) to exist for the class.

For example, the class **Scene** defined for video-editing software is completeonly if it contains all attributes and methods that can reasonably be associatedwith the creation of a video scene. Sufficiency ensures that the designclass contains only those methods that are sufficient to achieve the intent ofthe class, no more and no less.

Primitiveness. Methods associated with a design class should be focusedon accomplishing one service for the class. Once the service has been implementedwith a method, the class should not provide another way to accomplishthe same thing.

For example, the class **VideoClip**for video-editingsoftware might have attributes start-point and end-point to indicate the startand end points of the clip (note that the raw video loaded into the systemmay be longer than the clip that is used). The methods, *setStartPoint()* and*setEndPoint(),* provide the only means for establishing start and end pointsfor the clip.

High cohesion. A cohesive design class has a small, focused set of responsibilitiesand single-mindedly applies attributes and methods to implementthose responsibilities.

For example, the class **VideoClip**might contain a set ofmethods for editing the video clip. As long as each method focuses solely onattributes associated with the video clip, cohesion is maintained.

COHESION:

1.Coincidental cohesion
2.Logical cohesion
3.Temporal cohesion
4.Procedural cohesion
5.Communication cohesion
6.Sequential cohesion
7.Functional cohesion

Low coupling. Within the design model, it is necessary for design classes tocollaborate with one another. However, collaboration should be kept to anacceptable minimum. If a design model is highly coupled (all design classescollaborate with all other design classes), the system is difficult to implement,to test, and to maintain over time. In general, design classes within a subsystemshould have only limited knowledge of other classes.

This restriction,called the *Law of Demeter* [Lie03], suggests that a method should only sendmessages to methods in neighboring classes.

COUPLING:

1.Content coupling
2.Common coupling
3.Control coupling
4.Stomp coupling
5.Data coupling

Transform mapping & Transaction mapping

A mapping technique, called *structured design,* is often characterized as adata flow-oriented design method because it provides a convenient transition from adata flow diagram to software architecture.

The transition from informationflow (represented as a DFD) to program structure is accomplished as part of a sixstepprocess:

(1) The type of information flow is established,
(2) Flow boundaries areindicated,
(3) The DFD is mapped into the program structure,
(4) Control hierarchy isdefined,
(5) The resultant structure is refined using design measures and heuristics, and
(6) The architectural description is refined and elaborated.

Transform Mapping

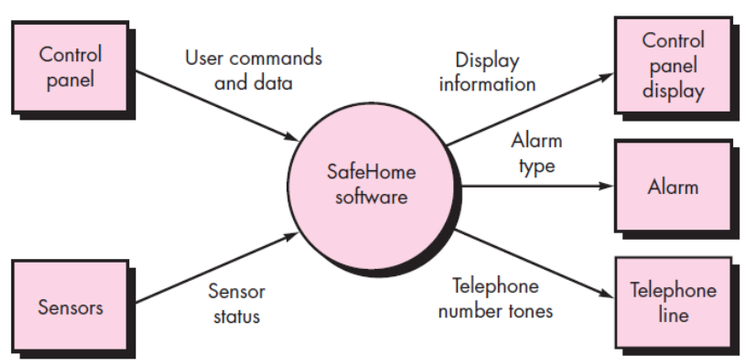
Transform mapping is a set of design steps that allows a DFD with transform flowharacteristics to be mapped into a specific architectural style. To illustrate thisapproach, we again consider the *SafeHome*security function.10 One element of theanalysis model is a set of data flow diagrams that describe information flow within the security function. To map these data flow diagrams into a software architecture,you would initiate the following design steps:

Step 1. Review the fundamental system model.

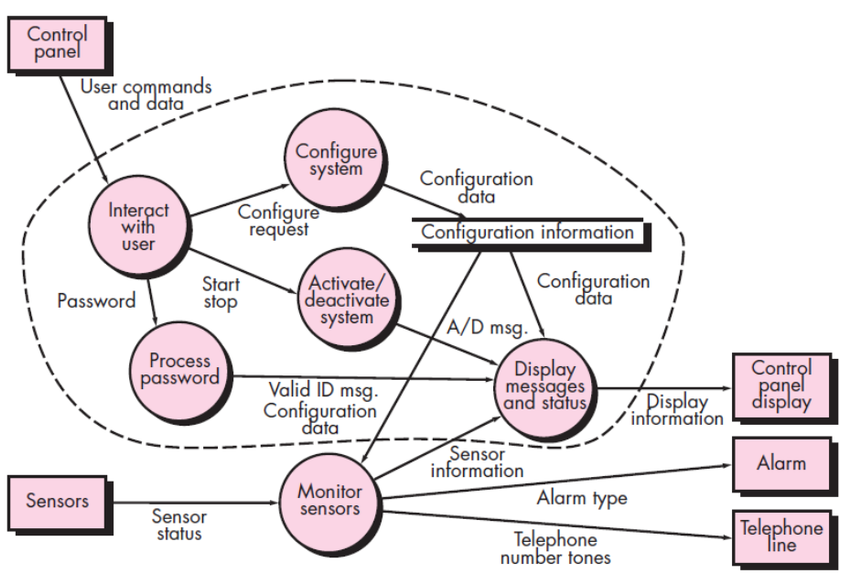
The fundamental system modelor context diagram depicts the security function as a single transformation, representingthe external producers and consumers of data that flow into and out of thefunction. Figure 9.10 depicts a level 0 context model, and Figure 9.11 shows refined data flow for the security function.

FIGURE 9.10

Context-level DFD for the *SafeHome* security function



Level 1 DFD for the *SafeHome* security function



Step 2. Review and refine data flow diagrams for the software. Informationobtained from the requirements model is refined to produce greater detail. Forexample, the level 2 DFD for *monitor sensors* (Figure 9.12) is examined, and a level 3data flow diagram is derived as shown in Figure 9.13. At level 3, each transform inthe data flow diagram exhibits relatively high cohesion.

FIGURE 9.12
Level 2 DFD that refines the *monitor sensors* transform

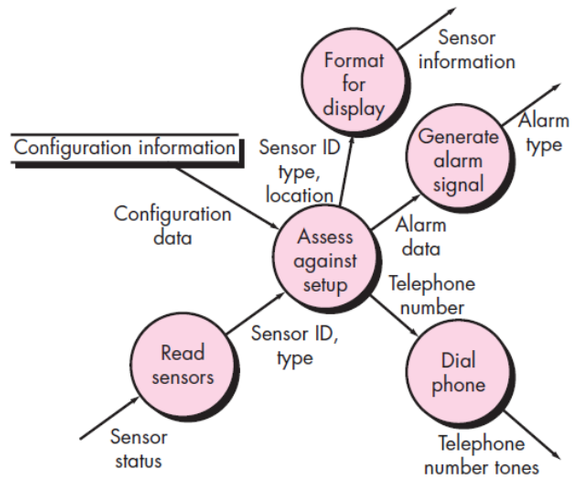
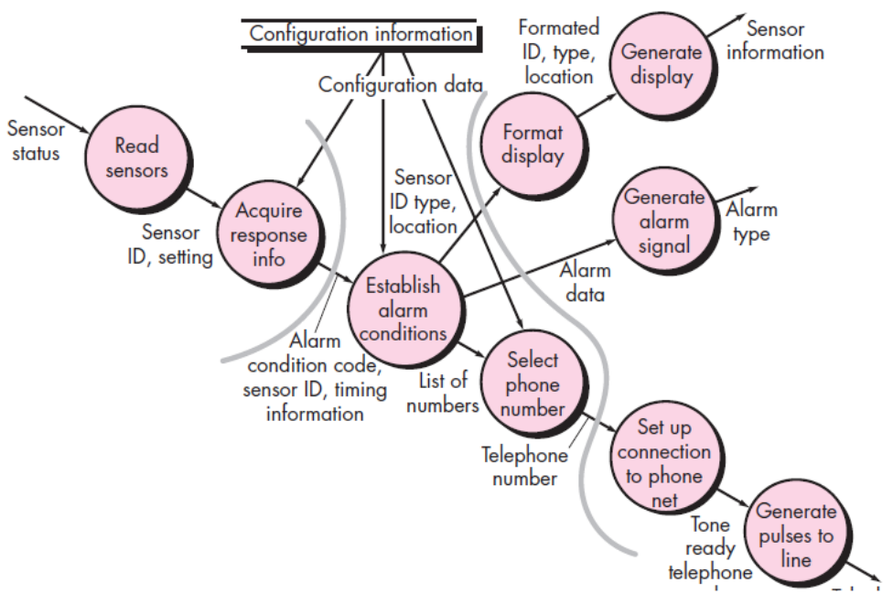


FIGURE 9.13 Level 3 DFD for *monitor sensors* with flow boundaries



Step 3. Determine whether the DFD has transform or transaction flow11characteristics. Evaluating the DFD (Figure 9.13), we see data entering the softwarealong one incoming path and exiting along three outgoing paths. Therefore, anoverall transform characteristic will be assumed for information flow.

Step 4. Isolate the transform center by specifying incoming and outgoingflow boundaries. Incoming data flows along a path in which information isconverted from external to internal form; outgoing flow converts internalized datato external form. Incoming and outgoing flow boundaries are open to interpretation.

That is, different designers may select slightly different points in the flow as boundarylocations. In fact, alternative design solutions can be derived by varying theplacement of flow boundaries. Although care should be taken when boundaries areselected, a variance of one bubble along a flow path will generally have little impacton the final program structure.

Flow boundaries for the example are illustrated as shaded curves running verticallythrough the flow in Figure 9.13. The transforms (bubbles) that constitute thetransform center lie within the two shaded boundaries that run from top to bottomin the figure. An argument can be made to

readjust a boundary (e.g., an incomingflow boundary separating *read sensors* and *acquire response info* could be proposed).

The emphasis in this design step should be on selecting reasonable boundaries,rather than lengthy iteration on placement of divisions.process implied by a transform performs a single, distinct function that can beimplemented as a component in the *SafeHome*software. Therefore, the DFD inFigure 9.13 contains sufficient detail for a "first cut" at the design of architecture forthe *monitor sensors* subsystem, and we proceed without further refinement.

Step 5. Perform "first-level factoring."The program architecture derived usingthis mapping results in a top-down distribution of control. *Factoring* leads to aprogram structure in which top-level components perform decision making and lowlevelcomponents perform most input, computation, and output work. Middle-levelcomponents perform some control and do moderate amounts of work.

When transform flow is encountered, a DFD is mapped to a specific structure(a call and return architecture) that provides control for incoming, transform, andoutgoing information processing. This first-level factoring for the *monitor sensors*subsystem is illustrated in Figure 9.14. A main controller (called *monitor sensorsexecutive*) resides at the top of the program structure and coordinates the followingsubordinate control functions:

- An incoming information processing controller, called *sensor input controller,*coordinates receipt of all incoming data.

- A transform flow controller, called *alarm conditions controller,* supervises alloperations on data in internalized form (e.g., a module that invokes variousdata transformation procedures).

- An outgoing information processing controller, called *alarm output controller,*coordinates production of output information.

Although a three-pronged structure is implied by Figure 9.14, complex flows inlarge systems may dictate two or more control modules for each of the genericcontrol functions described previously. The number of modules at the first levelshould be limited to the minimum that can accomplish control functions and stillmaintain good functional independence characteristics.

Step 6. Perform "second-level factoring."Second-level factoring is accomplishedby mapping individual transforms (bubbles) of a DFD into appropriatemodules within the architecture. Beginning at the transform center boundary andmoving outward along incoming and then outgoing paths, transforms are mappedinto subordinate levels of the software structure. The general approach to secondlevelfactoring is illustrated in Figure 9.15.

Although Figure 9.15 illustrates a one-to-one mapping between DFD transformsand software modules, different mappings frequently occur. Two or even three bubblescan be combined and represented as one component, or a single bubble maybe expanded to two or more components.

Step 7. Refine the first-iteration architecture using design heuristics forimproved software quality.

A first-iteration architecture can always be refined byapplying concepts of functional independence (Chapter 8). Components are explodedor imploded to produce sensible factoring, separation of concerns, good cohesion,minimal coupling, and most important, a structure that can be implemented withoutdifficulty, tested without confusion, and maintained without grief.

Refinements are dictated by the analysis and assessment methods describedbriefly in Section 9.5, as well as practical considerations and common sense. Thereare times, for example, when the controller for incoming data flow is totally unnecessary,when some input processing is required in a component that is subordinateto the transform controller, when high coupling

due to global data cannot beavoided, or when optimal structural characteristics cannot be achieved. Softwarerequirements coupled with human judgment is the final arbiter.

DESIGN DOCUMENTATION

- The *Design Specification* addresses different aspects of the design model and is completed as the designer refines his representation of the software.
- the overall scope of the design effort is described. Much of the information presented here is derived from the *System Specification* and the analysis model (*Software Requirements Specification*).
- the data design is specified. Database structure, any external file structures, internal data structures, and a cross reference that connects data objects to specific files are all defined.
- The architectural design indicates how the program architecture has been derivedfrom the analysis model.
- The structure charts are used to represent the module hierarchy (if applicable).

The design of external and internal program interfaces is represented and a detailed design of the human/machine interface is described. In some cases, a detailed prototype of a GUI may be represented.

Components—separately addressable elements of software such as subroutines, functions, or procedures—are initially described with an English-language processing narrative. The processing narrative explains the procedural function of a component (module). Later, a procedural design tool is used to translate the narrative into a structured description.

The Design Specification contains a requirements cross reference.
The purpose of this cross reference (usually represented as a simple matrix) is

(1) To establish that all requirements are satisfied by the software design and

(2) To indicate which components are critical to the implementation of specific requirements.

The first stage in the development of test documentation is also contained in the design document.

Once program structure and interfaces have been established, we can develop guidelines for testing of individual modules and integration of the entire package. In some cases, a detailed specification of test procedures occurs in parallel with design. In such cases, this section may be deleted from the *Design Specification.* Design constraints, such as physical memory limitations or the necessity for a specialized external interface, may dictate special requirements for assembling or packaging of software.

Special considerations caused by the necessity for program overlay, virtual memory management, high-speed processing, or other factors may cause modification in design derived from information flow or structure. In addition, this section describes the approach that will be used to transfer software to a customer site.

The final section of the *Design Specification* contains supplementary data. Algorithm descriptions, alternative procedures, tabular data, excerpts from other documents, and other relevant information are presented as a special note or as a separate appendix. It may be advisable to develop a *Preliminary Operations/Installation Manual* and include it as an appendix to the design document.

IEEE defines software design documentation as 'a description of software created to facilitate analysis, planning, implementation, and decision-making.

This design description is -used as a medium for communicating software design information and can be considered as a blueprint or model of the system.

the most widely used standard is by IEEE, which acts as a general framework.

This general framework can be customized and adapted to meet the needs of a particular organization. This template consists of several sections, which are listed below.

1. **Scope:**
   - Identifies the release or version of the system being designed.
   - The system is divided into modules; the relationship between them and functionalities will be defined.
   - Every iteration of the SDD document describes and identifies the software modules to be added or changed in a release.
2. **References:**
   - Lists references (both hardware and software documents and manuals) used in the creation of the SDD that may be of use to the designer, programmer, user, or management personnel.

   - This document is also considered useful for the readers of the document. In this section, any references made to the other documents including references to related project documents, especially the SRS are also listed.

   - The existing software documentation (if any) is also listed.

3. **Definition:** Provides a glossary of technical terms used in the document along with their definitions.

4. **Purpose:** States the purpose of this document and its intended audience. This is meant primarily for individuals who will be implementing the system.

5. **Design description information content:** Consists of the following subsections.

A.)Introduction: Since SDD represents the software design that is to be implemented, it should describe the design entities into which the system has been partitioned along with their significant properties and relationships.

B.) Design entity:

It is a software design component that is different from other design entities in terms of structure and function.
The objective of creating design entities is to partition the system into a set of components that can be implemented and modified independently. Note that each design entity is assigned with a unique name and serves a specific purpose and function but all possess some common characteristics.

C.)Design entity attributes: They are properties of the design entity and provide some factual information regarding the entity. Every attribute has an attached description, which includes references and design considerations. The attributes and their associated information are listed in Table.

Table Attributes and Description

Attributes	Description
Identification	Identifies name of the entity. All the entities have a unique name.
Type	Describes the kind of entity. This specifies the nature of the entity.
Purpose	Specifies why the entity exists.
Function	Specifies what the entity does.
Subordinates	Identifies sub-ordinate entity of an entity.
Dependencies	Describes relationships that exist between one entity and other entities.
Interface	Describes how entities interact among themselves.
Resources	Describes elements used by the entity that are external to the design.
Processing	Specifies rules used to achieve the specified functions.
Data	Identifies data elements that form part of the internal entity.

6. Design description organization: Consists of the following subsection.

7. Design views: They describe the software design in a comprehensive manner so that the process of information access and integration is simplified. The design of software can be viewed in multiple ways and each design view describes a distinct aspect of the system. Table lists various design views and their attributes.

Table Design Views and their Description

Design View	Description	Attribute
Decomposition description	Partitions the system into design entities.	Identification, type, purpose, function, and subordinate
Dependency description	Describes relationships between entities.	Identification, type, purpose, dependencies, and resources
Interface description	Consists of list that is required by the stakeholders (designers, developers, and testers) in order to design entities.	Identification, function and interfaces
Detail description	Describes internal details of the design entity.	Identification, processing, and data

TEXT BOOK

1. R.S .Pressman , “Software Engineering”, Tata McGraw Hill.

REFERENCE BOOKS

1. Watts Humphrey, Managing the Software Process, Addison – Wesley Publishing Company, Reading MA,1989.

2. Watts Humphrey, Characterizing the Software Process : A Maturity Framework – IEEE Software, March 1988.Watts Humphrey,W.L.Sweet , A Method for accessing The Software Engineering

3. Capabilities of Contractors, Technical Report, CMU/SEI-87-TR-23,ESD/TR-87-186.

4. Software Engineering – I am sommerville 6th Edition